

CSC103-Programming Fundamentals

Math, Character and String Class

Math Functions

Java provides many useful methods in the **Math** class for performing common mathematical functions.

The Math Class

- Class constants:
 - π
 - e
- Class methods:
 - Trigonometric Methods
 - Exponent Methods
 - Rounding Methods
 - min, max, abs, and random Methods

Trigonometric Methods

TABLE 4.1 Trigonometric Methods in the Math Class

<i>Method</i>	<i>Description</i>
<code>sin(radians)</code>	Returns the trigonometric sine of an angle in radians.
<code>cos(radians)</code>	Returns the trigonometric cosine of an angle in radians.
<code>tan(radians)</code>	Returns the trigonometric tangent of an angle in radians.
<code>toRadians(degree)</code>	Returns the angle in radians for the angle in degree.
<code>toDegree(radians)</code>	Returns the angle in degrees for the angle in radians.
<code>asin(a)</code>	Returns the angle in radians for the inverse of sine.
<code>acos(a)</code>	Returns the angle in radians for the inverse of cosine.
<code>atan(a)</code>	Returns the angle in radians for the inverse of tangent.

Trigonometric Methods

- `sin(double a)`
- `cos(double a)`
- `tan(double a)`
- `acos(double a)`
- `asin(double a)`
- `atan(double a)`

Radians

`toRadians(90)`

Examples:

```
Math.sin(0) returns 0.0
```

```
Math.sin(Math.PI / 6)  
returns 0.5
```

```
Math.sin(Math.PI / 2)  
returns 1.0
```

```
Math.cos(0) returns 1.0
```

```
Math.cos(Math.PI / 6)  
returns 0.866
```

```
Math.cos(Math.PI / 2)  
returns 0
```

Exponent Methods

- **`exp(double a)`**
Returns e raised to the power of a .
- **`log(double a)`**
Returns the natural logarithm of a .
- **`log10(double a)`**
Returns the 10-based logarithm of a .
- **`pow(double a, double b)`**
Returns a raised to the power of b .
- **`sqrt(double a)`**
Returns the square root of a .

Examples:

`Math.exp(1)` returns 2.71

`Math.log(2.71)` returns 1.0

`Math.pow(2, 3)` returns 8.0

`Math.pow(3, 2)` returns 9.0

**`Math.pow(3.5, 2.5)` returns
22.91765**

`Math.sqrt(4)` returns 2.0

`Math.sqrt(10.5)` returns 3.24

Rounding Methods

- **double ceil(double x):**
 - x rounded up to its nearest integer. This integer is returned as a double value.
- **double floor(double x):**
 - x is rounded down to its nearest integer. This integer is returned as a double value.
- **double rint(double x):**
 - x is rounded to its nearest integer. If x is equally close to two integers, the even one is returned as a double.
- **int round(float x):**
 - Return (int) Math.floor(x+0.5).
- **long round(double x):**
 - Return (long) Math.floor(x+0.5).

Rounding Methods Examples

```
Math.ceil(2.1) returns 3.0
Math.ceil(2.0) returns 2.0
Math.ceil(-2.0) returns -2.0
Math.ceil(-2.1) returns -2.0
Math.floor(2.1) returns 2.0
Math.floor(2.0) returns 2.0
Math.floor(-2.0) returns -2.0
Math.floor(-2.1) returns -3.0
Math rint(2.1) returns 2.0
Math rint(2.0) returns 2.0
Math rint(-2.0) returns -2.0
Math rint(-2.1) returns -2.0
Math rint(2.5) returns 2.0
Math rint(-2.5) returns -2.0
Math.round(2.6f) returns 3
Math.round(2.0) returns 2
Math.round(-2.0f) returns -2
Math.round(-2.6) returns -3
```


min, max, and abs

- **max(a, b) and min(a, b)**
Returns the maximum or minimum of two parameters.
- **abs(a)**
Returns the absolute value of the parameter.
- **random()**
Returns a random `double` value in the range [0.0, 1.0).

Examples:

Math.max(2, 3) returns 3

**Math.max(2.5, 3) returns
3.0**

**Math.min(2.5, 3.6)
returns 2.5**

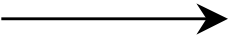
Math.abs(-2) returns 2

**Math.abs(-2.1) returns
2.1**

The random Method

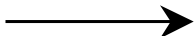
Generates a random double value greater than or equal to 0.0 and less than 1.0 ($0 \leq \text{Math.random()} < 1.0$).

Examples:

`(int) (Math.random() * 10)`  Returns a random integer between 0 and 9.

`50 + (int) (Math.random() * 50)`  Returns a random integer between 50 and 99.

In general,

`a + Math.random() * b`  Returns a random number between a and a + b, excluding a + b.

Character Data Type

- Computers use binary numbers internally
- Mapping a character to its binary representation is called *encoding*
- How characters are encoded is defined by an *encoding scheme*
- Java supports **Unicode** (16 bit character encoding and 65,536 characters are possible)
- A 16-bit Unicode takes **two bytes**
 - Preceded by `\u`, expressed in four hexadecimal digits
 - i.e. ranges from `\u0000` To `\uFFFF`

Casting between char and Numeric Types

```
int i = 'a'; // Same as int i = (int)'a';
```

```
char c = 97; // Same as char c = (char)97;
```

```
char ch = (char)65.25; // Decimal 65 is assigned to ch
```

```
System.out.println(ch); // ch is character A
```

- All numeric operators can be applied to char operands

```
int i = '2' + '3'; // (int)'2' is 50 and (int)'3' is 51
```

```
System.out.println("i is " + i); // i is 101
```

```
int j = 2 + 'a'; // (int)'a' is 97
```

```
System.out.println("j is " + j); // j is 99
```

```
System.out.println(j + " is the Unicode for character "
```

```
    + (char)j); // 99 is the Unicode for character c
```

```
System.out.println("Chapter " + '2');
```

i is 101

j is 99

99 is the Unicode for character c

Chapter 2

Comparing and Testing Characters

```
if (ch >= 'A' && ch <= 'Z')
```

```
    System.out.println(ch + " is an uppercase letter");
```

```
else if (ch >= 'a' && ch <= 'z')
```

```
    System.out.println(ch + " is a lowercase letter");
```

```
else if (ch >= '0' && ch <= '9')
```

```
    System.out.println(ch + " is a numeric character");
```

Methods in the Character Class

Method	Description
<code>isDigit(ch)</code>	Returns true if the specified character is a digit.
<code>isLetter(ch)</code>	Returns true if the specified character is a letter.
<code>isLetterOrDigit(ch)</code>	Returns true if the specified character is a letter or digit.
<code>isLowerCase(ch)</code>	Returns true if the specified character is a lowercase letter.
<code>isUpperCase(ch)</code>	Returns true if the specified character is an uppercase letter.
<code>toLowerCase(ch)</code>	Returns the lowercase of the specified character.
<code>toUpperCase(ch)</code>	Returns the uppercase of the specified character.

String Type

The String Type

- *A string is a sequence of characters.*
- String is an object that represents sequence of char values.
- The **char** type represents only one character
- An array of characters works same as Java string

```
char[] ch={'j','a','v','a','t','p','o','i','n','t'};  
String s = new String(ch);
```
- **String** is a predefined class in the Java library(java.lang)
- The String type is not a primitive type

The String Type

- Strings are objects in Java
- There are two ways to create String object:

- **By string literal**

```
String varName = "Some String";
```

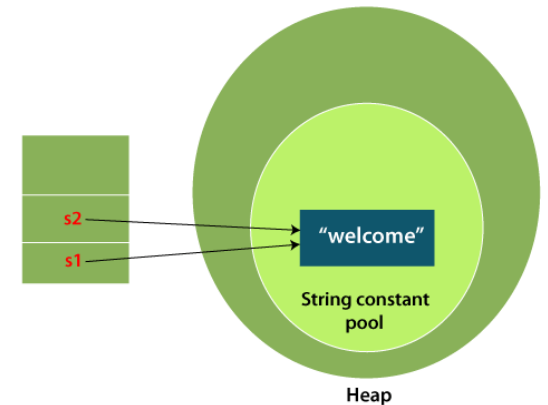
```
String s = "welcome";
```

- JVM first checks the

- String constant pool
- If the string already exists in the pool, a reference to the pooled instance is returned
- Otherwise, a new string instance is created and placed in the pool.

```
String s1 = "welcome";
```

```
String s2 = "welcome";
```



The String Type

- There are two ways to create String object:

- **By new keyword**

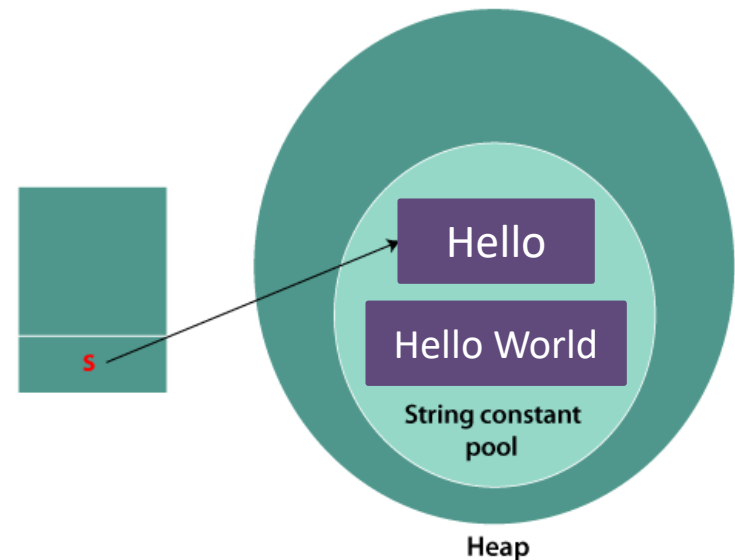
```
String varName = new String("Some String");  
String s = new String("welcome");
```

- JVM creates a new string object in normal heap memory
- In above example variable **s** will refer to the object in a heap

The String Type

- Immutable String
 - In java **String objects are immutable.** Immutable simply means unmodifiable or unchangeable.
 - Example

```
String s="Hello";  
s.concat(" World");  
System.out.println(s);
```



Simple Methods for **String** Objects

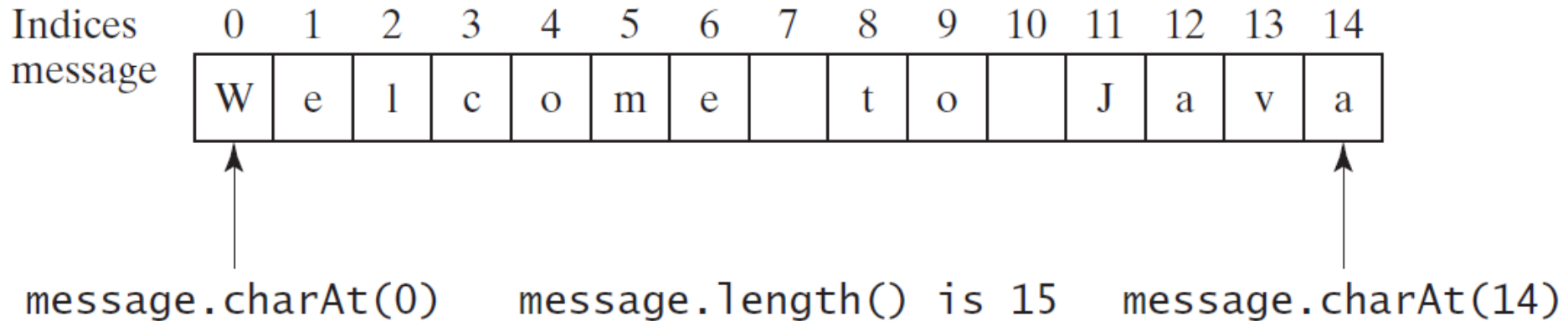
Method	Description
<code>length()</code>	Returns the number of characters in this string.
<code>charAt(index)</code>	Returns the character at the specified index from this string.
<code>concat(s1)</code>	Returns a new string that concatenates this string with string s1.
<code>toUpperCase()</code>	Returns a new string with all letters in uppercase.
<code>toLowerCase()</code>	Returns a new string with all letters in lowercase.
<code>trim()</code>	Returns a new string with whitespace characters trimmed on both sides.

Getting String Length

```
String message = "Welcome to Java";  
System.out.println("The length of " + message + " is " +  
message.length());
```

- The string appearing in program is called String literal
- Such String literals are instances of String class
- `System.out.println("Hello".length());`

Getting Characters from a String



```
String message = "Welcome to Java";
```

```
System.out.println("The first character in message is " + message.charAt(0));
```

String Concatenation

- `String s3 = s1.concat(s2);` or `String s3 = s1 + s2;`

// Three strings are concatenated

- `String message = "Welcome " + "to " + "Java";`

// String Chapter is concatenated with number 2

- `String s = "Chapter" + 2;` // s becomes Chapter2

// String Supplement is concatenated with character B

- `String s1 = "Supplement" + 'B';` // s1 becomes SupplementB
- The augmented `+=` operator can also be used for string concatenation
- Example:

`String message = "Welcome to Java"`

`message += " and Java is fun" //output: Welcome to Java and Java is fun`

String Concatenation

- + operator can also concatenate a number with a string in SOP function
 - In this case number is converted into the string and then concatenated.
- If $i = 1$ and $j = 2$, what is the output of the following statement?
`System.out.println("i + j is " + i + j);`
- Output: "i + j is 12"

```
System.out.println("1" + 1);  
System.out.println('1' + 1);  
System.out.println("1" + 1 + 1);  
System.out.println("1" + (1 + 1));  
System.out.println('1' + 1 + 1);
```


Converting Strings

`"Welcome".toLowerCase()` returns a new string, `welcome`.

`"Welcome".toUpperCase()` returns a new string, `WELCOME`.

`" Welcome ".trim()` returns a new string, `Welcome`.

Reading a String from the Console

```
Scanner input = new Scanner(System.in);
```

```
System.out.print("Enter three words separated by spaces: ");
```

```
String s1 = input.next();
```

```
String s2 = input.next();
```

```
String s3 = input.next();
```

```
System.out.println("s1 is " + s1);
```

```
System.out.println("s2 is " + s2);
```

```
System.out.println("s3 is " + s3);
```

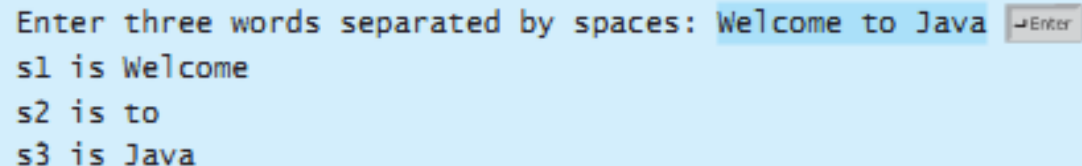
- The next() method reads a string that *ends* with a whitespace character
- nextLine() method to read an entire line of text that ends with the ENTER key is pressed

```
Scanner input = new Scanner(System.in);
```

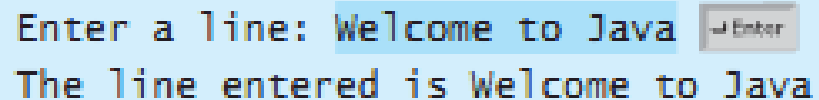
```
System.out.println("Enter a line: ");
```

```
String s = input.nextLine();
```

```
System.out.println("The line entered is " + s);
```



```
Enter three words separated by spaces: Welcome to Java Enter  
s1 is Welcome  
s2 is to  
s3 is Java
```



```
Enter a line: Welcome to Java Enter  
The line entered is Welcome to Java
```

Comparing Strings

<i>Method</i>	<i>Description</i>
<code>equals(s1)</code>	Returns true if this string is equal to string <code>s1</code> .
<code>equalsIgnoreCase(s1)</code>	Returns true if this string is equal to string <code>s1</code> ; it is case insensitive.
<code>compareTo(s1)</code>	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than <code>s1</code> .
<code>compareToIgnoreCase(s1)</code>	Same as <code>compareTo</code> except that the comparison is case insensitive.
<code>startsWith(prefix)</code>	Returns true if this string starts with the specified prefix.
<code>endsWith(suffix)</code>	Returns true if this string ends with the specified suffix.
<code>contains(s1)</code>	Returns true if <code>s1</code> is a substring in this string.

Comparing Strings

```
if (string1 == string2)
    System.out.println("string1 and string2 are the same object");
else
    System.out.println("string1 and string2 are different objects");
```

the == operator checks only whether string1 and string2 refer to the same object; it does not tell you whether they have the same contents.

equals(s2): The equals method returns true if two strings are equal and false if they are not.

```
String s1 = "Welcome to Java";
String s2 = "Welcome to Java";
String s3 = "Welcome to C++";
System.out.println(s1.equals(s2)); // true
System.out.println(s1.equals(s3)); // false
```

Comparing Strings

`compareTo(s1)`

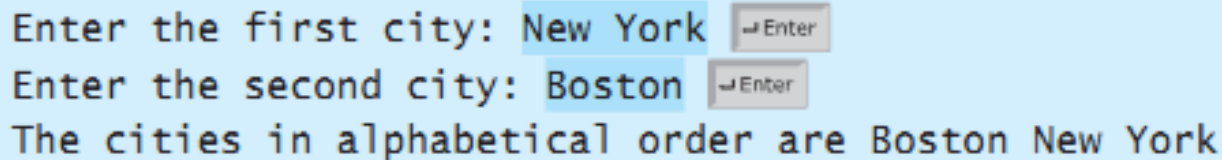
The method returns the **value 0** if **s1** is equal to **s2**, a **value less than 0** if **s1** is less than **s2**, and a **value greater than 0** if **s1** is greater than **s2**.

```
String s1 = "Welcome to Java";  
String s2 = "Welcome to Java";  
String s3 = "Welcome to C++";  
System.out.println (s1.compareTo(s2)); // Both are equal->0  
System.out.println (s1.compareTo(s3)); // s1 is greater than s3-> 7 ASCII of J=74 and C=67 (74-67)  
System.out.println (s3.compareTo(s1)); // s3 less than s1 -> -7 ASCII of J=74 and C=67 (67-74)
```

```
"Welcome to Java".startsWith("We") returns true.  
"Welcome to Java".startsWith("we") returns false.  
"Welcome to Java".endsWith("va") returns true.  
"Welcome to Java".endsWith("v") returns false.  
"Welcome to Java".contains("to") returns true.  
"Welcome to Java".contains("To") returns false.
```

Example

```
Scanner input = new Scanner(System.in);  
// Prompt the user to enter two cities  
System.out.print("Enter the first city: ");  
String city1 = input.nextLine();  
System.out.print("Enter the second city: ");  
String city2 = input.nextLine();  
if (city1.compareTo(city2) < 0)  
    System.out.println("The cities in alphabetical order are " + city1 + " " + city2);  
else  
    System.out.println("The cities in alphabetical order are " + city2 + " " + city1);
```



The screenshot shows a Java program running in a terminal. The first prompt is "Enter the first city:", followed by the user input "New York" and an "Enter" button. The second prompt is "Enter the second city:", followed by the user input "Boston" and another "Enter" button. The final output line is "The cities in alphabetical order are Boston New York".

```
Enter the first city: New York Enter  
Enter the second city: Boston Enter  
The cities in alphabetical order are Boston New York
```

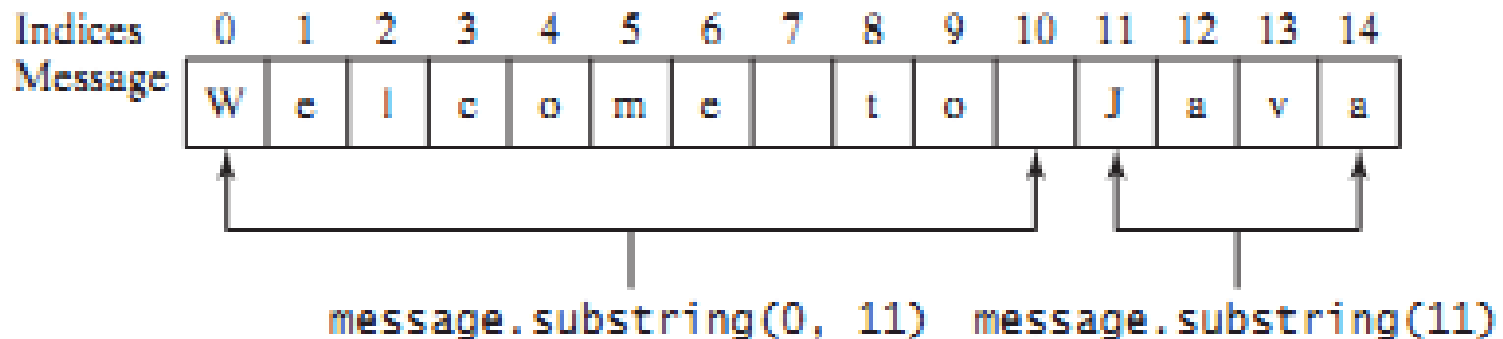
Obtaining Substrings

substring(beginIndex)

Returns this string's substring that begins with the character at the specified beginIndex and extends to the end of the string

substring(beginIndex, endIndex)

Returns this string's substring that begins at the specified beginIndex and extends to the character at index endIndex – 1. Note that the character at endIndex is not part of the substring.



```
String message = "Welcome to Java";
```

```
System.out.println(message.substring(3)); //come to Java
```

```
System.out.println(message.substring(11,message.length())); //Java
```

Finding a Character or a Substring in a String

<i>Method</i>	<i>Description</i>
<code>index(ch)</code>	Returns the index of the first occurrence of <code>ch</code> in the string. Returns <code>-1</code> if not matched.
<code>indexOf(ch, fromIndex)</code>	Returns the index of the first occurrence of <code>ch</code> after <code>fromIndex</code> in the string. Returns <code>-1</code> if not matched.
<code>indexOf(s)</code>	Returns the index of the first occurrence of string <code>s</code> in this string. Returns <code>-1</code> if not matched.
<code>indexOf(s, fromIndex)</code>	Returns the index of the first occurrence of string <code>s</code> in this string after <code>fromIndex</code> . Returns <code>-1</code> if not matched.
<code>lastIndexOf(ch)</code>	Returns the index of the last occurrence of <code>ch</code> in the string. Returns <code>-1</code> if not matched.
<code>lastIndexOf(ch, fromIndex)</code>	Returns the index of the last occurrence of <code>ch</code> before <code>fromIndex</code> in this string. Returns <code>-1</code> if not matched.
<code>lastIndexOf(s)</code>	Returns the index of the last occurrence of string <code>s</code> . Returns <code>-1</code> if not matched.
<code>lastIndexOf(s, fromIndex)</code>	Returns the index of the last occurrence of string <code>s</code> before <code>fromIndex</code> . Returns <code>-1</code> if not matched.

Finding a Character or a Substring in a String

indexOf()

"Welcome to Java".indexOf('W') returns 0.

"Welcome to Java".indexOf('o') returns 4.

"Welcome to Java".indexOf('o', 5) returns 9.

"Welcome to Java".indexOf("come") returns 3.

"Welcome to Java".indexOf("Java", 5) returns 11.

"Welcome to Java".indexOf("java", 5) returns -1.

lastIndexOf()

"Welcome to Java".lastIndexOf('W') returns 0.

"Welcome to Java".lastIndexOf('o') returns 9.

"Welcome to Java".lastIndexOf('o', 5) returns 4.

"Welcome to Java".lastIndexOf("come") returns 3.

"Welcome to Java".lastIndexOf("Java", 5) returns -1.

"Welcome to Java".lastIndexOf("Java") returns 11.

Conversion between Strings and Numbers

- To convert a string into an **int** value, use the **Integer.parseInt** method

```
int intValue = Integer.parseInt(intString);
```

– where *intString* is a numeric string such as "123"

- To convert a string into a **double** value, use the **Double.parseDouble** method

```
double doubleValue = Double.parseDouble(doubleString);
```

– where *doubleString* is a numeric string such as "123.45"